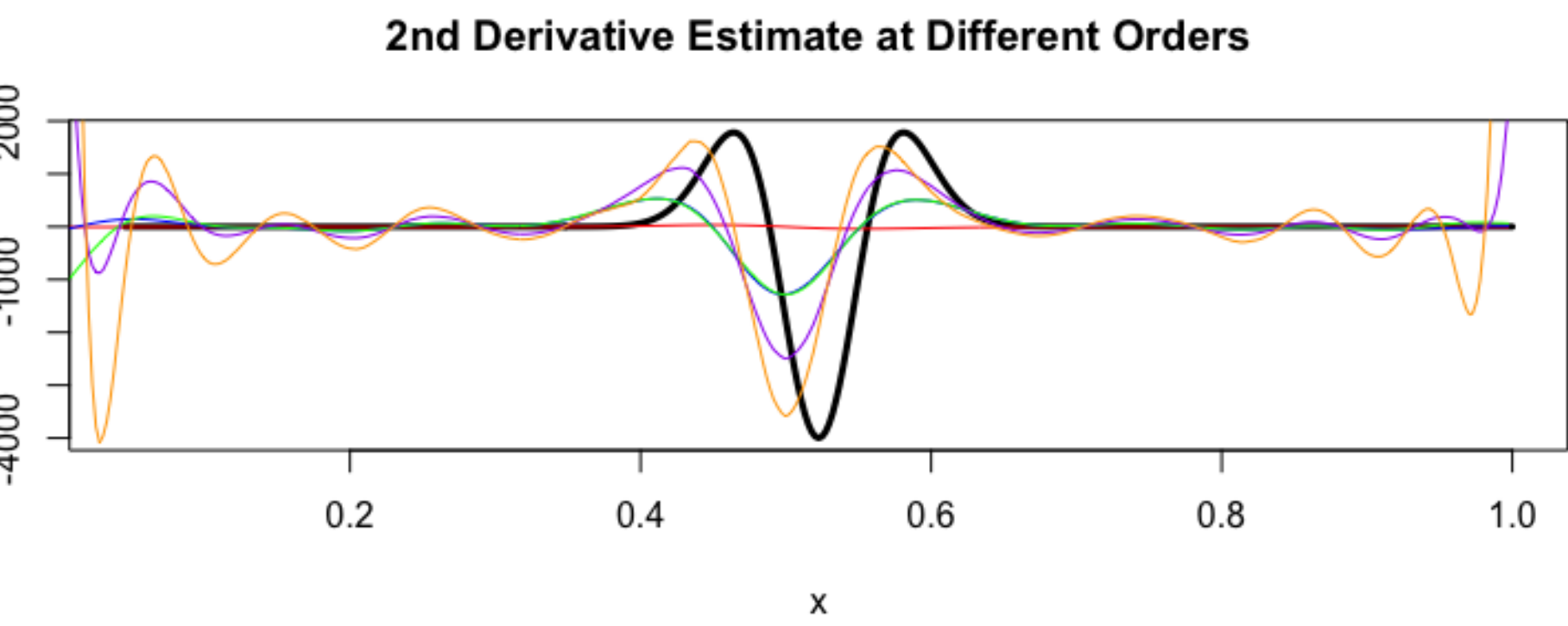


Lasso Regularization on Local Polynomial Regression for Derivative Estimation

Aidan Elliott, Lengyi Han, John Braun

Research Question

The derivatives of a regression can be difficult to estimate as there is no data points to regress with. Local polynomial regression (locpoly) provides an attractive way to estimate local derivatives, but its results vary significantly with polynomial order p and the optimal p frequently changes along the domain. We look to use lasso regularization to choose an optimal order p along the domain while reducing the variance of the estimators via shrinkage.



The second derivative estimation of local polynomial regression (n=500) using different order polynomials. The truth is black, and the different colors use order p=2,3,5,7. Each estimate uses the same bandwidth. It shows the importance of choosing optimal hyperparameters in local regression.

Derivative Use Cases

- Physics (particle velocity acceleration)
- Economics (modeling flexible market)
- Plugin Bandwidth Estimation
- Spline Knot Selection

Other Methods

- Empirical Derivatives
- Local Polynomial Regression
- Splines

Theory

Local polynomial regression involves approximating a point of interest via a Taylor polynomial. The equation's structure allows it to be fit as a weighted least squares regression. The betas will contain all the derivative estimates up to chosen order p.

$$m(x) \approx m(x_0) + m'(x_0)(x - x_0) + \dots + \frac{m^{(p)}(x_0)}{p!}(x - x_0)^p$$

$$X = \begin{bmatrix} 1 & (x_1 - x_0) & (x_1 - x_0)^2 & \dots & (x_1 - x_0)^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x_0) & (x_n - x_0)^2 & \dots & (x_n - x_0)^p \end{bmatrix}$$

$$\hat{\beta} = (X^T W X)^{-1} X^T W Y \quad \hat{m}^{(p)}(x_0) = j! \hat{\beta}_j$$

Local linear or cubic fits are generally sufficient for estimating the regression, but that is not necessarily the case for the derivatives. However, a bias-variance tradeoff for selecting a higher order polynomial p prevents us from fitting an arbitrarily large order. The estimator's asymptotic variance is dependent on a matrix of kernel moments up to order 2p.

$$Var(\hat{m}^{(v)}(x_0)|X_1, \dots, X_n) = e_{v+1}^T S^{-1} S^* S^{-1} e_{v+1} \frac{v!^2 \sigma^2(x_0)}{f(x_0) n h^{1+2v}} \{1 + o_P(1)\}$$

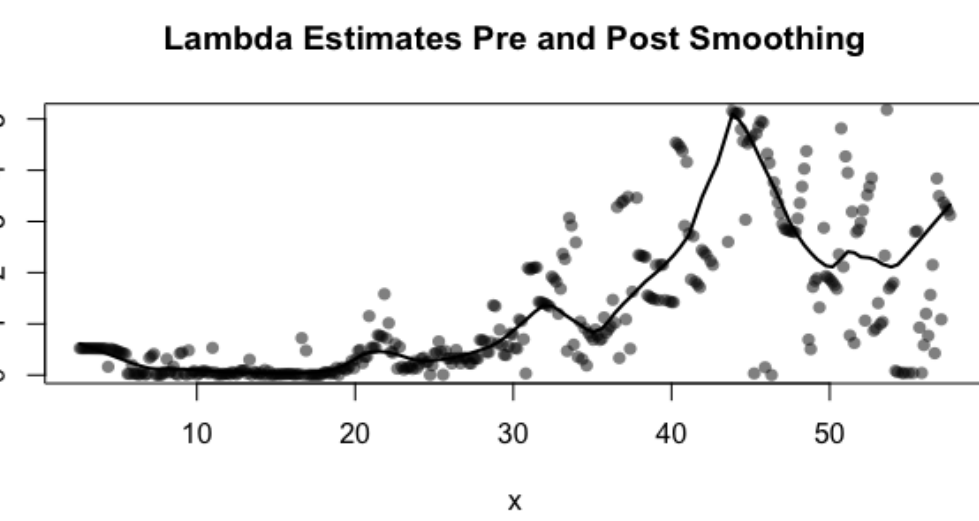
$$S = \begin{bmatrix} u_0 & u_1 & \dots & u_p \\ u_1 & u_2 & \dots & u_{p+1} \\ \vdots & \vdots & \ddots & \vdots \\ u_p & u_{p+1} & \dots & u_{2p} \end{bmatrix} \quad S^* = \begin{bmatrix} v_0 & v_1 & \dots & v_p \\ v_1 & v_2 & \dots & v_{p1} \\ \vdots & \vdots & \ddots & \vdots \\ v_p & v_{p+1} & \dots & v_{2p} \end{bmatrix} \quad \text{where} \quad \begin{aligned} u_j &= \int w^j K(u) du \\ v_j &= \int w^j K^2(u) du \end{aligned}$$

What happens if some terms below p were also excluded from the regression?

Lasso-LPR Implementation

Since the suitable level of shrinking will change along with the curvature of the function, a regularization parameter lambda must be calculated at each point of interest (POI). The glmnet library calculates the optimal lambda using cross validation.

At each POI, build the design matrix X up to a desired order p. We used a p equal to 9 to show the lasso was able to regularize itself despite a p too large for any traditional local regression fit. Weights were calculated using a normal kernel, and the bandwidth was decided based on a rule of thumb standard for traditional local polynomial regression (Fan and Gijbels 1996). We initially used a larger bandwidth and 10-fold cross validation to calculate the lambda at each POI. However, the estimates of lambda were noisy, causing the output to be jagged as neighboring points used very different shrink levels. When plotting a curve of the lambdas, it looked more like a scatterplot than a smooth shrinkage curve. Our solution was to apply LOESS smoothing, then refitting the model with the new lambda estimates to get a smoother output.

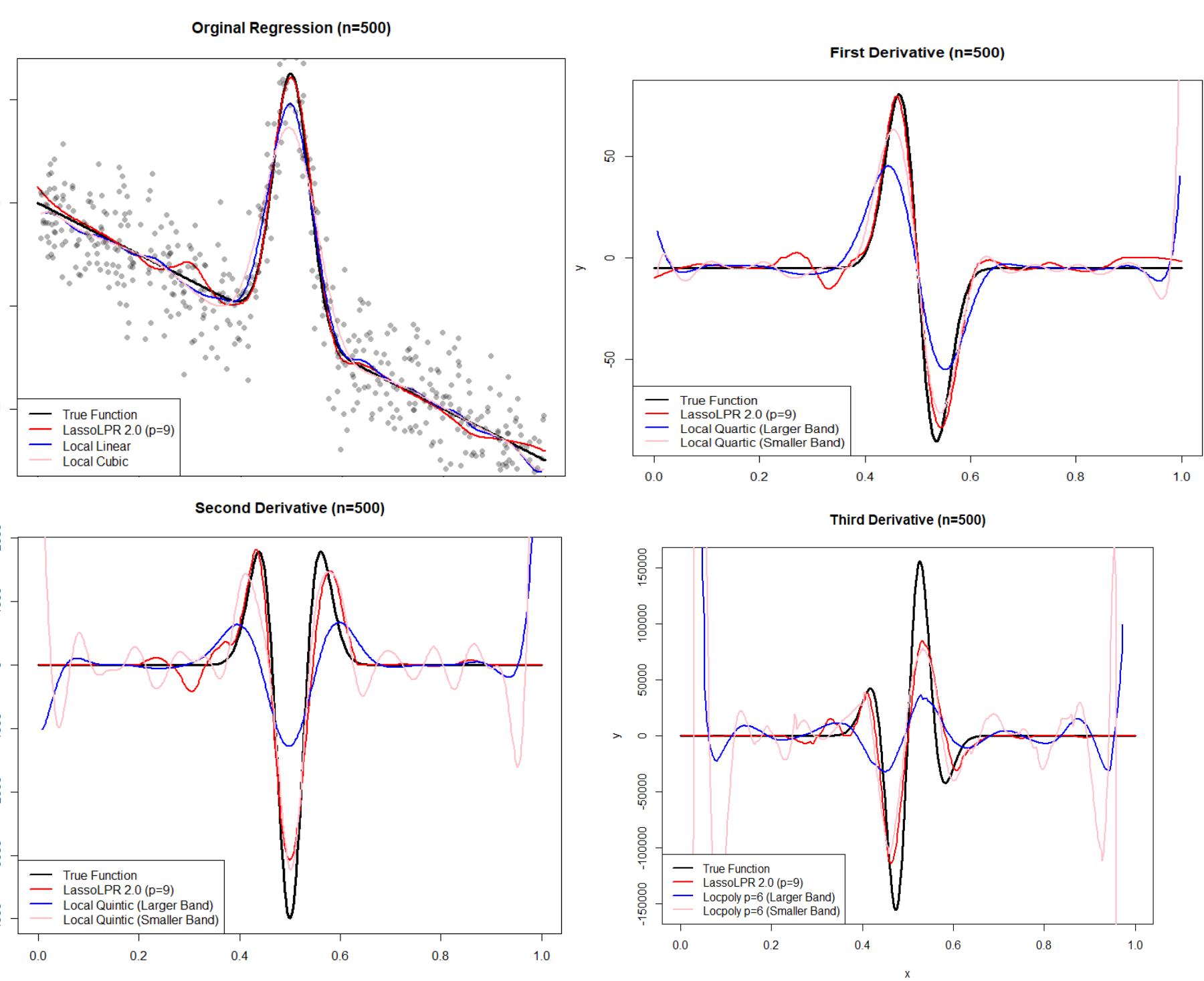


Plot of 10-fold CV lambda estimates from the Case Study Regression. The 401 lambda estimates are smoothed using LOESS with span 0.35 and refit to produce an output smoother regression output.

An example of a second derivative Lasso-LPR fit before and after using the smoothed lambdas. The black line shows the unreliability of using the Lasso without smoothing

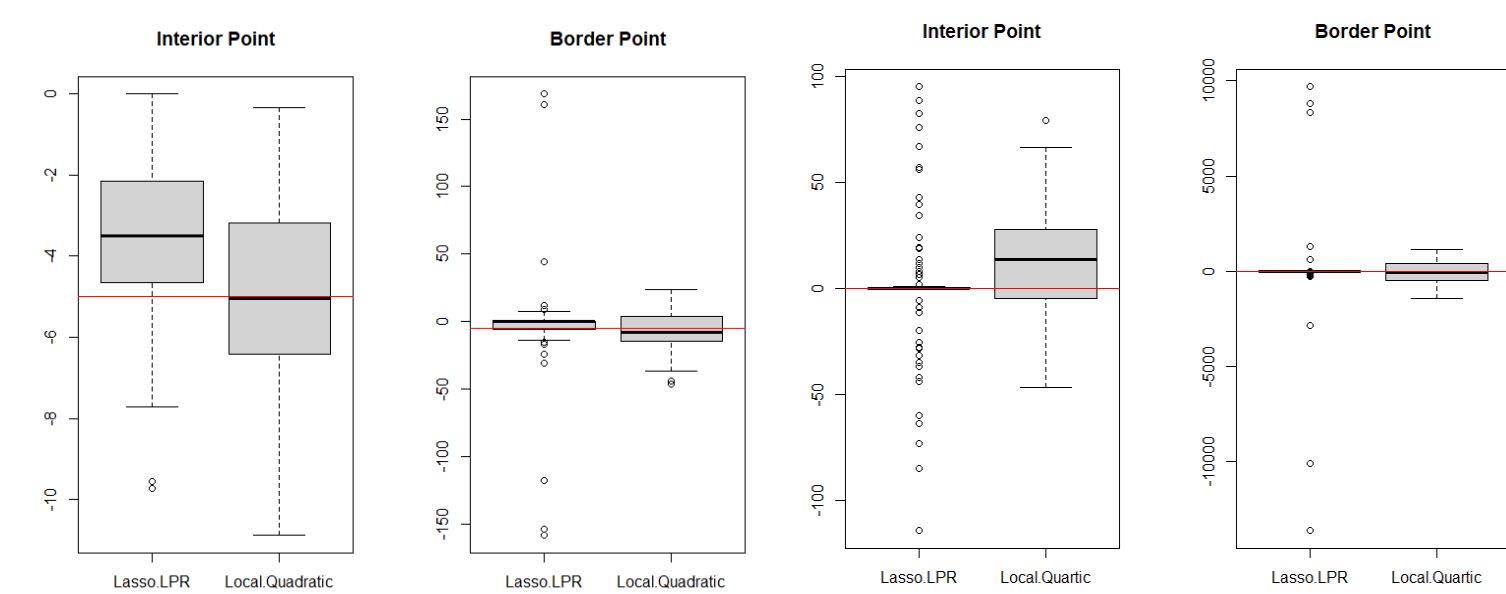
Since the lasso does not have a closed form solution, the cross validation proves to be an expensive calculation to do for every POI. As the lambdas were being smoothened anyway, we could reduce the compute time significantly by performing the cross validation at less points (usually 90% less) then extrapolating a smooth output to all POIs. We also changed the calculation to be LOOCV with a smaller bandwidth to make sure the calculation focused locally. The changes proved to decrease compute time by over 50% and slightly improve performance.

Simulated Example



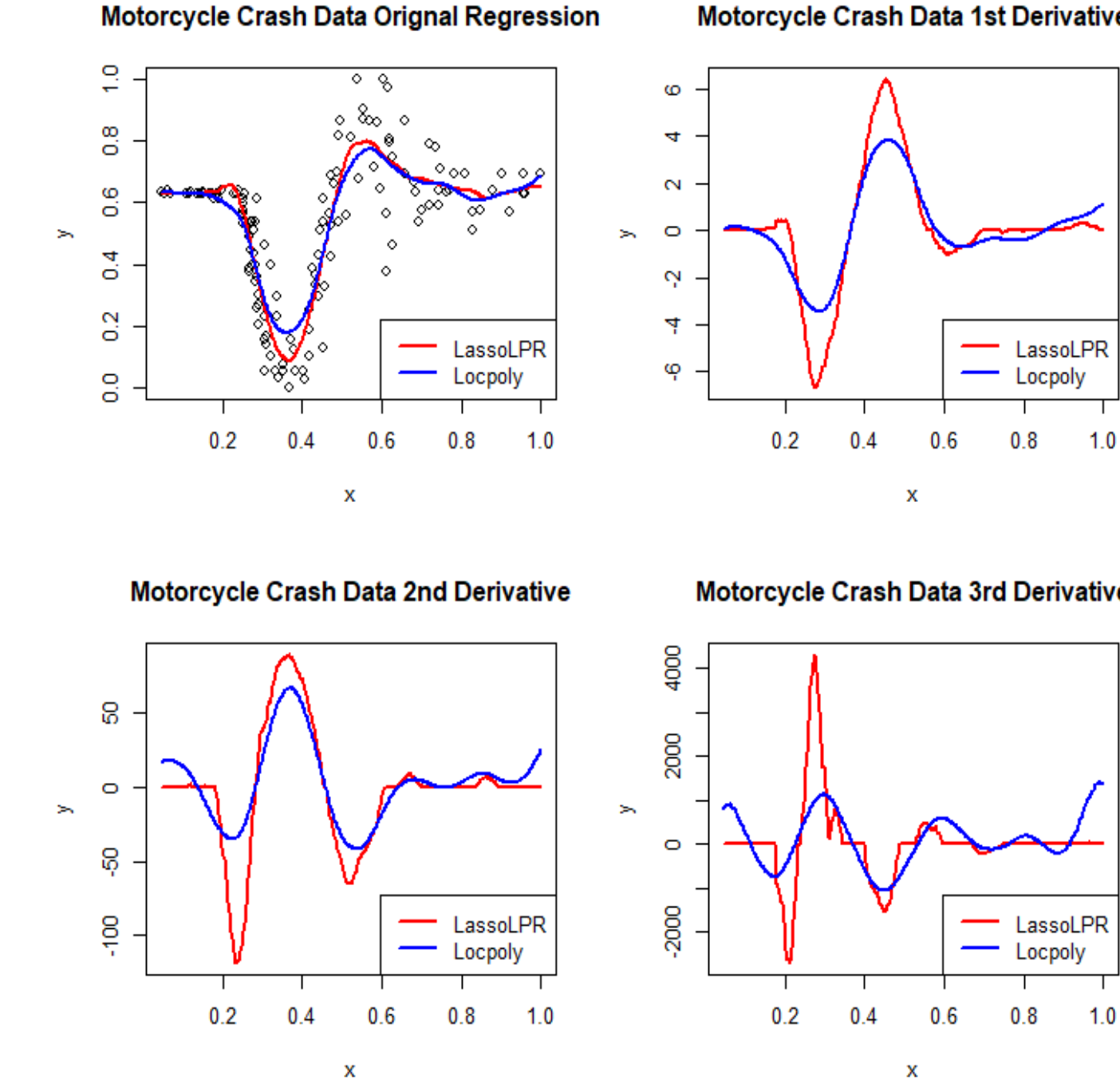
An example function used to Lasso-LPR (p=9, red) with local regressions (blue and pink). A dataset was generated with n=500. The original regression and the first three derivatives were plotted with the true value in black. A fixed rule of thumb bandwidth was used for all methods. The Lasso-LPR captured the peaks very well and behaved favorably on the flatter slopes.

Average Performance



Boxplot demonstrating the performance of border and interior points in the first and second derivative over 100 simulations. The Lasso-LPR method proved to add a little bias but dropped the variance significantly more. However, the Lasso-LPR proved to produce some large outliers. This came from the scenario where the CV technique estimates a lambda near zero on the border. This fit a polynomial of degree 9, causing it to "blow up". Modifying the method to enforce larger lambdas on the border would greatly increase the robustness of the estimator.

Case Study

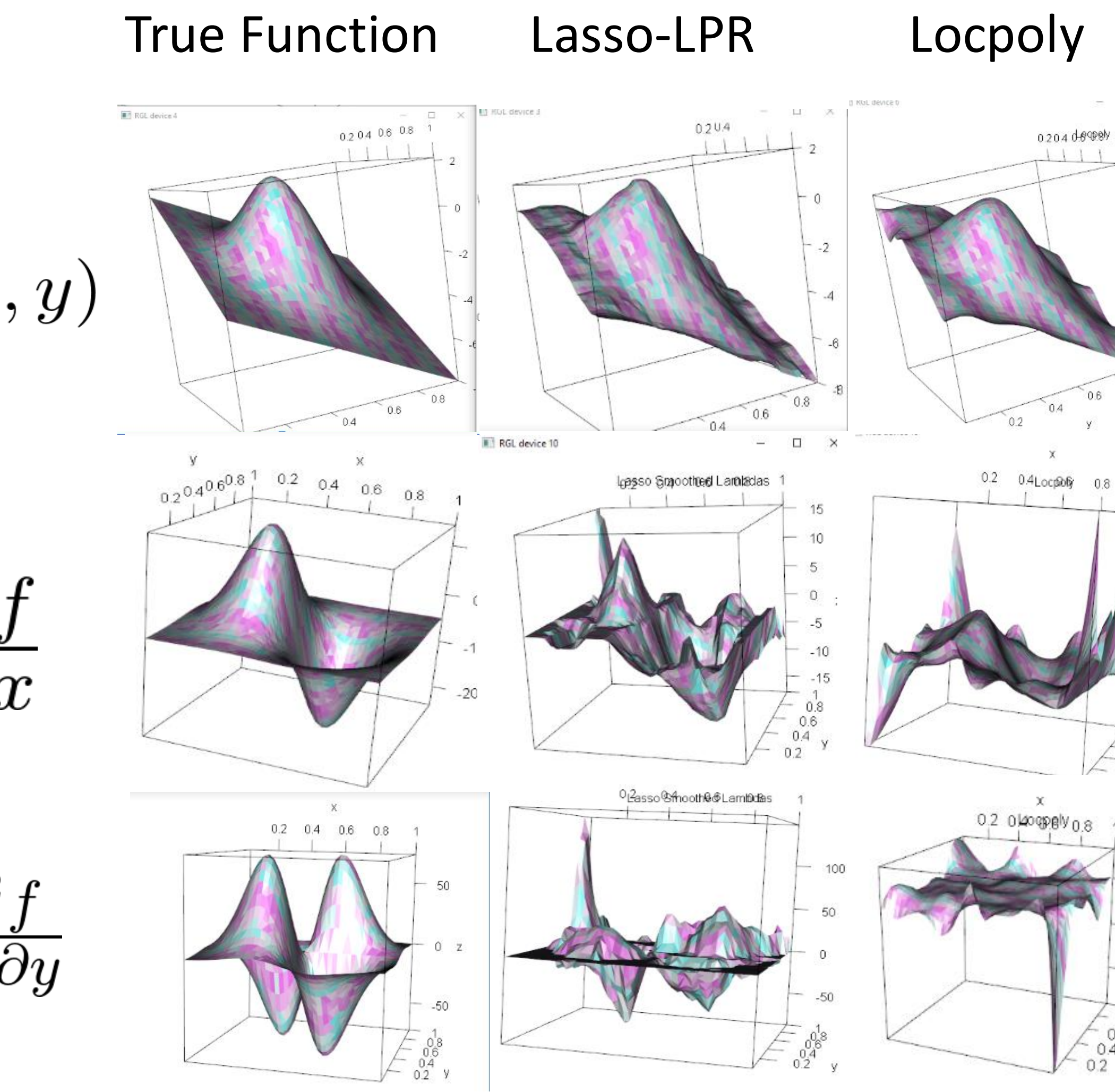


This dataset is the acceleration of a dummy's head during a car crash. There is no "truth" to compare the output with, but the Lasso-LPR (red, p=9) was able to model the sudden change at impact and flatten to zero after the crash better than local regression.

Higher Dimensions

As dimensions increase, the number of Taylor polynomial terms increase even quicker. More opportunity to use the lasso for variable selection.

$$\text{Number of terms in m-dimensional polynomial of degree p: } \frac{(m+p)!}{m!p!}$$



An example of a 3D regression with n=250. Lasso-LPR conducted with p=4. Plot shows that Lasso-LPR is partially more jagged, but grasps the shape of the peaks better, and still flattens out well around the border. Local polynomials struggle to grasp the shape of the derivatives, especially beyond the first one.

Conclusion/Future Work

- The Lasso operator can improve the performance of higher order derivative estimates while maintaining the original regression.
- Lasso-LPR can model sharp change points better than traditional local regression methods
- The method shows promise to work in higher dimensional derivative estimation.
- Calculating the penalization parameter lambda is noisy and expensive. A rule of thumb method is being explored to reduce complexity.
- The robustness of the penalty parameter can be improved by forcing lambda to be high towards the borders.

References

Dai, W., Tong, T., & Genton, M. G. (2016). Optimal estimation of derivatives in nonparametric regression. *Journal of Machine Learning Research*, 17(164), 1-25.

Fan, J., & Gijbels, I. (1995). Adaptive order polynomial fitting: bandwidth robustification and bias reduction. *Journal of Computational and Graphical Statistics*, 4(3), 213-227.

Fan, J., Gijbels, I., Hu, T. C., & Huang, L. S. (1996). A study of variable bandwidth selection for local polynomial regression. *Statistica Sinica*, 113-127

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1), 267-288.

Seifert, B., & Gasser, T. (2000). Data adaptive ridging in local polynomial regression. *Journal of Computational and Graphical Statistics*, 9(2), 338-360.

